

TECHNICAL DESIGN DOCUMENT (TTD)

CodeSphere – CSE Coding Learning & Assessment Platform

Project: CodeSphere – CSE Coding Learning & Assessment Platform

Initial Learning Language: C Programming

Target Users: CSE Coding Club students and administrators

Expected Concurrent Usage: Approximately 60 students

1. Technical Overview

CodeSphere will use a modern web architecture. The frontend provides the learning dashboard and coding interface. FastAPI provides backend APIs and business logic. MongoDB stores application data. Judge0 executes untrusted C programs in an isolated execution environment.

2. Technology Stack

Layer	Technology	Purpose
Frontend	React + TypeScript + Vite	Single-page web application
UI	Tailwind CSS	Responsive interface styling
Code Editor	Monaco Editor	VS Code-like browser editor
Backend	Python FastAPI	REST APIs and application logic
Authentication	JWT + password hashing	User authentication and authorization
Database	MongoDB Atlas	Application and submission data
Code Execution	Judge0	Sandboxed compilation/execution of C programs
Frontend Hosting	Vercel	Static frontend deployment
Backend Hosting	Render/Railway/VPS	FastAPI deployment
Video Resources	YouTube links/embed	Learning references

3. High-Level Architecture

Student/Admin Browser → React Frontend → FastAPI Backend → MongoDB Atlas.

For code execution: React Frontend → FastAPI → Judge0 → execution result → FastAPI → MongoDB → React Frontend.

The FastAPI backend acts as the trusted application layer. The frontend should not directly control privileged database operations or hidden test-case logic.

4. Authentication Design

JWT (JSON Web Token) will be used to authenticate API requests. After successful login, the backend issues a signed access token containing the user identity and role. Protected endpoints validate the token before processing requests.

- Passwords stored only as secure hashes.
- JWT access tokens should have expiration times.
- Role-based authorization distinguishes students from administrators.
- Protected endpoints identify the authenticated user from the validated token rather than trusting a user ID supplied by the client.

5. Frontend Architecture

Area	Main Components
Authentication	Login form, route protection, session handling
Student Dashboard	Progress cards, modules, upcoming rounds, recent activity
Learning	Module list, topic detail, YouTube references, completion tracking
Practice	Problem statement, Monaco Editor, input/output panel, Run/Submit
Coding Round	Timer, problem navigation, autosave, warning dialog, final submit
Results	Score summary, verdicts, solved problems, leaderboard
Admin	Problem management, learning resources, round creation, result analytics

6. Backend API Design

Method	Endpoint	Purpose
POST	/api/auth/login	Authenticate user and issue JWT
GET	/api/auth/me	Return authenticated user profile
GET	/api/learning/modules	Get C learning modules
GET	/api/learning/topics/{id}	Get topic details/resources
GET	/api/problems	List available problems
GET	/api/problems/{id}	Get problem statement
POST	/api/code/run	Execute code for permitted test input
POST	/api/code/submit	Submit code for evaluation
GET	/api/rounds	Get available rounds
POST	/api/rounds/{id}/start	Start assessment session
POST	/api/rounds/{id}/autosave	Save current code
POST	/api/rounds/{id}/submit	Submit assessment
GET	/api/results	Get student results
POST	/api/admin/problems	Create problem
POST	/api/admin/rounds	Create coding round
GET	/api/admin/results	View round/student results

7. Database Design

Recommended MongoDB collections:

Collection	Key Fields
users	name, email, passwordHash, registerNumber, role, class
learning_modules	title, description, order, language
learning_topics	moduleId, title, content, videoUrl, order
problems	title, description, examples, constraints, difficulty, language
test_cases	problemId, input, expectedOutput, visibility
coding_rounds	title, duration, problemIds, startTime, endTime, status
round_sessions	roundId, studentId, startedAt, status, remainingTime
submissions	studentId, problemId, code, language, verdict, score, timestamps
autosaves	sessionId, problemId, code, updatedAt
activity_events	sessionId, eventType, timestamp, metadata

8. Code Execution Workflow

1. Student writes C code in Monaco Editor.
2. Student clicks Run or Submit.
3. Frontend sends code and problem/session context to FastAPI.
4. FastAPI validates the authenticated user and assessment state.
5. Backend creates a Judge0 submission using the configured C language runtime.
6. Judge0 compiles and executes the code in its sandbox.
7. Backend receives execution status and output.
8. For assessment submissions, backend evaluates configured test cases and calculates score.
9. Final verdict and metadata are stored in MongoDB.
10. Frontend displays the result.

9. Judge0 Integration Design

- Use Judge0 only through the backend service.
- Do not expose private API credentials in the frontend.
- Set execution limits such as time and memory according to the Judge0 deployment/API configuration.
- Use asynchronous polling or callback-compatible workflow where supported.
- Store normalized verdicts such as Accepted, Wrong Answer, Compilation Error, Runtime Error and Time Limit Exceeded.
- Keep hidden test cases inaccessible to the student frontend.
- Use retries and timeout handling for transient service failures.

10. Coding Round and Auto-Submit Design

When a student starts a coding round, the backend creates a round session with server-side timestamps. The frontend displays the remaining time but the server remains the authority for whether the session is active or expired.

- Autosave editor content periodically and before important navigation events.
- On timer expiry, submit the latest saved/current code.
- Use the Page Visibility API and window focus events to record when the assessment page is hidden or loses focus.
- If the round policy explicitly enables it, a qualifying visibility violation can trigger automatic submission and session locking.
- Because browser focus detection is not perfect, this feature should be treated as a policy/control mechanism rather than a guarantee of cheating detection.
- Show students a clear assessment warning before they start.

11. Concurrency Design for 60 Students

The application should be designed so that 60 simultaneous students do not cause the main FastAPI process to compile code. Dashboard traffic is handled by the web/API layer, while execution requests are delegated to Judge0.

- Use asynchronous FastAPI endpoints for I/O-heavy external calls.
- Add rate limiting to prevent accidental repeated Run requests.
- Debounce or limit rapid execution requests from a single student.
- Use background processing/queue patterns if submission bursts become significant.
- Pre-test the system with simulated concurrent users before a real coding event.
- Use a hosting plan that remains awake during scheduled assessments.
- Monitor backend response time, database connections and Judge0/API limits.

12. Deployment Architecture

Recommended initial deployment: React frontend on Vercel, FastAPI backend on a reliable cloud service, MongoDB Atlas as the managed database, and Judge0 as the isolated code execution service.

Component	Deployment Responsibility
Vercel	Serve React frontend
Backend Host	Run FastAPI APIs and application logic
MongoDB Atlas	Store persistent data
Judge0	Compile and execute C programs safely
Environment Variables	Store database URI, JWT secret and external API credentials

13. Security Considerations

- Never execute arbitrary student code directly on the FastAPI host.
- Hash passwords using a modern password hashing algorithm.
- Keep JWT secret keys and Judge0 credentials in environment variables.
- Validate all request payloads using Pydantic models.

- Apply role-based access checks to admin APIs.
- Do not expose hidden test cases to the frontend.
- Use HTTPS in production.
- Log important assessment events without storing unnecessary sensitive data.

14. Testing Strategy

Test Type	Examples
Unit Testing	Authentication, score calculation, output normalization
API Testing	Login, protected endpoints, submissions, admin controls
Integration Testing	FastAPI ↔ MongoDB ↔ Judge0 workflow
Frontend Testing	Timer, editor state, autosave, route protection
Load Testing	Simulate approximately 60 concurrent students
Security Testing	Unauthorized access, invalid JWT, role bypass attempts
User Acceptance Testing	Conduct a pilot coding round with a small student group

15. Development Phases

Phase	Deliverables
Phase 1	Project setup, MongoDB, authentication, student/admin roles
Phase 2	C learning modules, YouTube resources, progress tracking
Phase 3	Problem bank, Monaco Editor, Judge0 Run/Submit
Phase 4	Coding rounds, timer, autosave, visibility event policy
Phase 5	Results, leaderboard, admin analytics
Phase 6	Security review, load testing for 60 users, production deployment

16. Recommended Project Structure

Frontend: src/components, src/pages, src/services, src/hooks, src/context.

Backend: app/main.py, routes/, models/, schemas/, services/, database/, core/security.py.

Key backend services: auth_service, judge_service, submission_service, round_service and leaderboard_service.

17. Future Enhancements

- Support C++, Python, Java and additional languages.
- AI coding assistant for hints and error explanations.
- Personalized learning recommendations.
- Badges, streaks and gamification.
- Advanced analytics for weak-topic detection.
- Email/notification reminders for upcoming coding rounds.

- More sophisticated assessment monitoring policies.